

RAPPORT DE TP

Systemes distribués

Architecture, défis et sécurité d'un système de gestion documentaire distribué

Module : Applications Réparties et Cybersécurité — Séance 5
1ère année Cybersécurité — 2025/2026

1. Introduction et contexte

Cette séance portait sur les concepts fondamentaux des systèmes distribués : absence d'horloge globale, pannes partielles, théorème CAP, modèles d'architecture (client-serveur, peer-to-peer, microservices) et implications cybersécurité (Zero Trust, surfaces d'attaque, menaces spécifiques). Le TP guidé s'appuie sur un projet fil rouge, le Système de Gestion Documentaire Distribué (SGDD) : une plateforme permettant à une organisation de stocker, rechercher et gérer des documents de manière distribuée, supportant plusieurs centaines d'utilisateurs simultanés, avec des contraintes de disponibilité et de sécurité (documents confidentiels, audit d'accès, conformité RGPD).

Ce rapport présente les trois livrables demandés : la schématisation de l'architecture du SGDD (TP5.1), l'analyse des défis distribués par composant (TP5.2), et la cartographie des surfaces d'attaque avec un threat modeling simplifié de type STRIDE (TP5.3 et lab sécurité).

2. TP5.1 — Schématisation de l'architecture distribuée

2.1 Composants identifiés

Cinq services ont été identifiés pour le SGDD, chacun avec une responsabilité claire et une frontière bien définie :

Composant	Responsabilité	Technologie envisagée
Auth Service	Authentification (login), émission de tokens JWT, gestion des rôles	Python (FastAPI), PostgreSQL
Stockage Service	CRUD des documents (upload, download, suppression), métadonnées	Python, stockage objet (MinIO), MongoDB
Recherche Service	Indexation et recherche plein texte dans les documents	Python, Elasticsearch
API Gateway	Point d'entrée unique, routage, rate limiting, validation des tokens	Traefik ou Python custom
Notification Service	Alertes lors d'uploads, modifications, partages (optionnel)	Python, Redis Pub/Sub, email

2.2 Architecture logique

Le schéma ci-dessus représente l'architecture logique retenue : le client passe systématiquement par l'API Gateway, qui vérifie les tokens JWT, applique le rate limiting et route la requête vers le service concerné (Auth, Stockage, Recherche ou Notification). Chacun de ces quatre services dispose de sa propre base de données dédiée (pattern database per service) : PostgreSQL pour les utilisateurs, MongoDB + stockage objet pour les documents, Elasticsearch pour l'index de recherche, et Redis pour la file de messages utilisée par les notifications. La synchronisation entre le Stockage et la Recherche se fait de façon asynchrone via un événement `document_uploaded` publié dans la file Redis, et tous les services envoient leurs logs vers un système de journalisation et de traces centralisé, utilisé pour l'audit et l'observabilité.

2.3 Protocoles de communication entre services

Interaction	Protocole	Synchrone / asynchrone
Client → API Gateway	HTTPS / REST (JSON)	Synchrone
API Gateway → Auth Service	HTTP REST interne (vérification JWT)	Synchrone
API Gateway → Stockage Service	HTTP REST interne	Synchrone
API Gateway → Recherche Service	HTTP REST interne	Synchrone
Stockage Service → Recherche Service	Événement document_uploaded via Redis Pub/Sub	Asynchrone
Tous les services → Logs centralisés	Envoi de logs structurés (JSON) via syslog/agent	Asynchrone (fire-and-forget)

2.4 Architecture physique

Pour un environnement de laboratoire, chaque service (API Gateway, Auth, Stockage, Recherche, Notification) est packagé dans son propre conteneur Docker, ce qui correspond directement aux compétences acquises lors des séances précédentes (Docker, puis Kubernetes). En développement, ces conteneurs peuvent tourner sur une seule VM (docker compose) ; en production, ils seraient répartis sur plusieurs nœuds d'un cluster Kubernetes, avec un Pod par instance de service et la possibilité de répliquer indépendamment chaque service selon sa charge (par exemple plusieurs répliques du Recherche Service si les requêtes de recherche sont fréquentes, alors qu'une seule instance d'Auth peut suffire avec une réplique de secours).

Les bases de données (PostgreSQL, MongoDB, Elasticsearch, Redis) sont déployées sur des nœuds séparés des services applicatifs, dans un sous-réseau privé non exposé directement à l'extérieur, conformément au principe de segmentation réseau. Seul l'API Gateway dispose d'une exposition réseau vers l'extérieur (port HTTPS).

2.5 Réponses au gabarit de questions

1. Nombre de services et frontières : 5 services (API Gateway, Auth, Stockage, Recherche, Notification). Chaque frontière correspond à un domaine métier distinct (authentification, persistance des documents, indexation/recherche, notification), ce qui respecte le principe de responsabilité unique et permet un déploiement et une mise à l'échelle indépendants.
2. Protocoles : toutes les communications synchrones (Gateway ↔ services) utilisent HTTP REST avec des payloads JSON. La synchronisation Stockage → Recherche utilise un message asynchrone publié dans une file Redis, ce qui découple les deux services dans le temps.
3. Placement des bases de données : chaque service possède sa propre base de données (database per service) — PostgreSQL pour Auth, MongoDB/MinIO pour Stockage, Elasticsearch pour Recherche, Redis pour la file de Notification. Aucune base n'est mutualisée entre services, ce qui évite un couplage fort au niveau des données et

permet à chaque service de choisir la technologie de stockage la mieux adaptée à son usage.

4. Appels en parallèle : lors d'une recherche, si le résultat doit être enrichi par des informations utilisateur (par exemple le nom de l'auteur de chaque document), l'appel au service Recherche et l'appel au service Auth/Users pour résoudre les identités des auteurs peuvent être effectués en parallèle (via `asyncio.gather` côté Gateway) plutôt que séquentiellement, réduisant la latence perçue.
5. Chemin critique par opération : pour l'upload, le chemin critique est Client → Gateway → Auth (vérification JWT) → Stockage (écriture du fichier et des métadonnées) ; l'indexation par Recherche et la notification sont hors chemin critique car traitées de manière asynchrone. Pour le download, le chemin critique est Client → Gateway → Auth → Stockage. Pour la recherche, le chemin critique est Client → Gateway → Auth → Recherche.

2.6 Justification des choix (synthèse)

L'architecture retenue privilégie une approche microservices avec API Gateway, car le SGDD doit supporter plusieurs centaines d'utilisateurs simultanés et des contraintes de sécurité fortes (documents confidentiels, audit RGPD), ce qui justifie l'isolation des responsabilités et la possibilité de scaler indépendamment le service le plus sollicité (Recherche ou Stockage selon l'usage). Le découplage entre Stockage et Recherche via une file de messages permet d'absorber les pics d'upload sans bloquer l'utilisateur en attendant l'indexation, au prix d'une cohérence éventuelle pour la recherche (un document tout juste uploadé peut ne pas apparaître immédiatement dans les résultats). Le pattern database per service évite qu'une panne ou une montée en charge d'un service n'impacte directement les données des autres. Enfin, le passage obligatoire par l'API Gateway centralise l'authentification et le rate limiting, tout en n'étant pas le seul point de contrôle de sécurité : conformément à Zero Trust, chaque service vérifiera également les tokens JWT de manière indépendante.

3. TP5.2 — Analyse des défis distribués par composant

Pour chaque service du SGDD, les défis de cohérence, de tolérance aux pannes et de latence ont été analysés, avec une proposition de solution architecturale s'appuyant sur les patterns vus en cours (cache, réplication, file de messages, fallback, circuit breaker).

Composant	Défi	Risque concret	Proposition de solution
Auth Service	Disponibilité	Si Auth tombe, personne ne peut se connecter → tout le système est bloqué	Réplication (2 instances actives), cache des tokens JWT validés côté API Gateway pour limiter les appels
Stockage Service	Cohérence	Un document uploadé mais pas encore indexé → invisible en recherche pendant un court délai	File de messages pour la synchronisation vers Recherche ; cohérence éventuelle acceptable pour ce cas d'usage
Recherche Service	Latence	L'indexation de gros documents peut être lente,	Indexation asynchrone via la file de messages ; cache

		bloquant le retour à l'utilisateur si elle est synchrone	des recherches fréquentes (Redis) pour réduire la charge sur Elasticsearch
API Gateway	Tolérance aux pannes	Si un service backend (ex: Recherche) tombe, les requêtes vers ce service échouent et peuvent saturer les threads du Gateway	Circuit breaker par service backend : si Recherche échoue N fois, le Gateway retourne immédiatement un message « service temporairement indisponible » sans attendre le timeout
Notification Service	Cohérence / latence	Si Notification est lent ou indisponible, cela ne doit pas retarder l'upload ni la réponse au client	Fallback : l'envoi de notification est entièrement asynchrone (file de messages) ; en cas d'échec, le message reste dans la file et est retraité plus tard sans bloquer le reste du système
Bases de données (toutes)	Tolérance aux pannes / disponibilité	Perte du nœud hébergeant une base de données → service correspondant indisponible et données potentiellement perdues	Réplication des bases critiques (PostgreSQL en réplication primaire/secondaire, MongoDB en replica set) avec failover automatique

3.1 Discussion des trade-offs

Chaque solution proposée introduit ses propres compromis. Le cache des tokens JWT côté Gateway réduit la charge sur Auth et améliore la latence, mais introduit un délai avant qu'une révocation de token (par exemple en cas de compromission d'un compte) ne soit effectivement prise en compte par le Gateway — un compromis entre performance et réactivité de sécurité, qui doit être borné par une durée de cache courte (par exemple quelques minutes). La file de messages entre Stockage et Recherche découple les deux services et absorbe les pics de charge, mais signifie qu'un document tout juste uploadé peut ne pas être immédiatement trouvable par la recherche : ce choix de cohérence éventuelle est acceptable pour un moteur de recherche documentaire, mais ne serait pas acceptable pour une opération financière. Le circuit breaker sur l'API Gateway protège le système d'une cascade de pannes si Recherche est en difficulté, mais signifie que pendant la période où le circuit est ouvert, les utilisateurs ne pourront pas effectuer de recherche, même si Recherche serait en réalité capable de traiter une requête isolée : on accepte une dégradation de service pour préserver la disponibilité globale. Enfin, la réplication des bases de données améliore la disponibilité et la tolérance aux pannes, mais complexifie la cohérence (lectures potentiellement légèrement obsolètes sur une réplique) et augmente le coût opérationnel (synchronisation, monitoring du replica set).

L'objectif de cette analyse n'est pas d'éliminer tous les risques — ce qui est impossible dans un système distribué — mais de choisir consciemment, pour chaque composant, le compromis entre cohérence, disponibilité et latence le plus adapté à son rôle dans le SGDD.

4. TP5.3 — Cartographie des surfaces d'attaque

Sept surfaces d'attaque ont été identifiées pour l'architecture du SGDD, avec pour chacune la menace principale, le contrôle proposé et une priorité (critique, élevée, moyenne).

Surface d'attaque	Menace principale	Contrôle proposé	Priorité
API externe (Gateway)	DDoS, injection, brute force sur l'authentification	Rate limiting, WAF, validation stricte des entrées, lockout après N échecs de connexion	Critique
Inter-services (Auth ↔ Stockage ↔ Recherche)	MITM, usurpation d'identité d'un service	mTLS entre tous les services, vérification de l'audience du JWT à chaque service	Critique
Bases de données (PostgreSQL, MongoDB, Elasticsearch)	Injection SQL/NoSQL, accès non autorisé	Requêtes paramétrées / ORM, credentials stockés dans un vault, bases dans un réseau privé non exposé	Critique
Interface d'administration	Accès non autorisé au monitoring ou au déploiement	VPN obligatoire pour l'accès admin, authentification multi-facteurs (MFA), RBAC dédié aux administrateurs	Élevée
Logs et traces	Fuite de données sensibles (tokens, données personnelles) dans les logs	Masquage des données sensibles (PII, tokens) avant écriture, accès restreint aux logs	Élevée
Secrets et configuration (clés API, certificats, mots de passe DB)	Fuite dans le code source, les variables d'environnement ou les images Docker	Stockage dans un vault (HashiCorp Vault), rotation automatique, scan des secrets en CI/CD	Critique
Dépendances externes (pip, images Docker de base)	Supply chain attack via un paquet ou une image compromise	pip-audit régulier, fichiers de verrouillage de versions (lock files), SBOM, revue des mises à jour	Moyenne

4.1 Mesures transverses Zero Trust

Au-delà de cette matrice par surface, l'architecture applique les principes Zero Trust de manière transversale : authentification et autorisation à chaque service (pas seulement au Gateway), principe de moindre privilège (par exemple le service Recherche dispose uniquement d'un accès en lecture à l'index Elasticsearch et n'a aucun accès à la base de Stockage), chiffrement systématique en transit (TLS côté externe, mTLS en interne) et au repos pour les documents confidentiels, segmentation réseau plaçant toutes les bases de données dans un sous-réseau privé, et vérification continue plutôt qu'une session de confiance permanente.

5. Lab Sécurité — Threat modeling et réponse à incident

5.1 Threat modeling STRIDE simplifié appliqué au SGDD

Catégorie STRIDE	Question posée	Exemple appliqué au SGDD
Spoofing (usurpation)	Un attaquant peut-il se faire passer pour un utilisateur ou un service légitime ?	Fabrication d'un faux JWT pour accéder aux documents d'un autre utilisateur
Tampering (altération)	Un attaquant peut-il modifier des données en transit ou au repos ?	Modification d'un document pendant son transfert entre le Stockage Service et le Recherche Service (MITM) si mTLS n'est pas en place
Repudiation (dénî)	Un utilisateur peut-il nier avoir effectué une action ?	Un utilisateur supprime un document et nie l'avoir fait, en l'absence de logs d'audit horodatés et signés
Information Disclosure (fuite d'info)	Des données sensibles peuvent-elles être exposées ?	Logs contenant des tokens JWT en clair, ou messages d'erreur détaillés exposés en production révélant la structure interne
Denial of Service	Le système peut-il être rendu indisponible ?	DDoS sur l'API Gateway, ou upload de fichiers volumineux saturant le Stockage Service
Elevation of Privilege	Un utilisateur peut-il obtenir des droits supérieurs ?	Modification du contenu d'un JWT pour passer du rôle reader à admin, si la signature n'est pas vérifiée correctement

5.2 Checklist Zero Trust Readiness appliquée au SGDD

L'architecture proposée a été évaluée par rapport aux principes Zero Trust :

- AuthN à chaque service : chaque service (Auth, Stockage, Recherche, Notification) vérifie indépendamment le token JWT, pas seulement l'API Gateway — conforme.
- AuthZ granulaire : les permissions sont vérifiées au niveau de chaque opération (lecture, écriture, suppression) via les rôles encodés dans le JWT — conforme par conception, à valider lors de l'implémentation.
- Moindre privilège : le service Recherche n'a accès qu'en lecture à son index Elasticsearch et n'a aucun accès direct à la base MongoDB du Stockage — conforme.
- Chiffrement en transit : TLS entre le client et le Gateway, mTLS prévu entre tous les services internes — à mettre en œuvre via un service mesh (Istio) si le système est déployé sur Kubernetes.
- Chiffrement au repos : les documents marqués confidentiels devront être chiffrés dans le stockage objet (MinIO) — point à formaliser dans les spécifications de Stockage Service.

- Segmentation réseau : toutes les bases de données (PostgreSQL, MongoDB, Elasticsearch, Redis) sont placées dans un sous-réseau privé non exposé — conforme dans l'architecture physique proposée.
- Logs d'audit : chaque accès à un document (qui, quoi, quand, depuis quelle IP) doit être journalisé vers le système de logs centralisé — prévu dans l'architecture, à détailler au niveau du format des logs.
- Rotation des secrets : les credentials des bases de données et les clés de signature JWT devront être rotés régulièrement (par exemple tous les 30 jours) via un vault — à mettre en place en amont du déploiement.
- Validation d'entrée : chaque service doit valider ses propres entrées (types, tailles, formats) sans faire confiance uniquement au Gateway — principe retenu pour l'implémentation.
- Limitation de débit : le rate limiting est prévu au niveau du Gateway, et devra également être appliqué sur les services critiques (Auth, Recherche) pour limiter l'impact d'un Gateway contourné.
- Scan de dépendances : exécution régulière de pip-audit (ou équivalent) en CI/CD pour les dépendances Python de chaque service — prévu dans le pipeline de build.
- Plan de réponse à incident : voir section 5.3 ci-dessous.

5.3 Scénario de compromission du Stockage Service : plan de réponse

Scénario : un attaquant exploite une vulnérabilité dans le Stockage Service et exécute du code arbitraire. Il peut potentiellement lire tous les documents, accéder à la base MongoDB, et appeler les autres services internes. Le plan de réponse suit les trois phases classiques confinement, éradication, rétablissement, suivies d'un post-mortem.

Phase 1 — Confinement

- Isoler immédiatement le Stockage Service du réseau (couper les règles réseau ou retirer le Pod du service mesh) pour empêcher la propagation latérale de l'attaque.
- Révoquer tous les tokens et credentials utilisés par ce service (certificat mTLS, clés d'accès à MongoDB et MinIO).
- Bloquer au niveau du Gateway et du service mesh tous les appels inter-services vers et depuis Stockage.
- Activer un mode dégradé : les autres services (Recherche, Notification) retournent un message 'service temporairement indisponible' pour les opérations liées au stockage, tandis que l'authentification continue de fonctionner.

Phase 2 — Éradication

- Analyser les logs du Stockage Service compromis : quand l'intrusion a-t-elle eu lieu, comment, et quelles données ont été accédées.
- Identifier la vulnérabilité exploitée (dépendance vulnérable connue, injection, exécution de code à distance).
- Corriger la vulnérabilité (mise à jour de dépendance, correctif applicatif).
- Scanner les autres services (Auth, Recherche, Notification) pour vérifier qu'ils n'ont pas également été compromis via les appels inter-services.
- Vérifier l'intégrité des documents stockés (modifications ou exfiltrations détectées via les logs d'accès).

Phase 3 — Rétablissement

- Redéployer le Stockage Service à partir d'une image Docker propre, sans restaurer l'instance compromise.
- Effectuer la rotation de tous les secrets : credentials MongoDB et MinIO, clés de signature JWT, certificats mTLS — y compris ceux des autres services, par précaution, car un service compromis peut avoir eu accès à des secrets partagés.
- Restaurer les données depuis une sauvegarde vérifiée si une altération a été constatée.
- Réactiver progressivement les connexions inter-services, en surveillant les comportements anormaux pendant 48 à 72 heures.

Post-mortem

- Rédiger un rapport d'incident détaillant la chronologie, l'impact et les actions menées.
- Identifier les causes profondes : pourquoi la compromission a-t-elle été possible, quelles alertes auraient dû se déclencher plus tôt.
- Mettre à jour les règles de segmentation réseau, les scans automatisés et les procédures de surveillance.
- Si des données personnelles ont été concernées, notifier les utilisateurs et les autorités compétentes dans le délai RGPD de 72 heures.

Ce scénario illustre un point essentiel : même avec une architecture Zero Trust bien conçue, un service peut être compromis. La rotation immédiate et exhaustive des secrets est critique, car un attaquant ayant récupéré les secrets d'un service compromis pourrait continuer à les utiliser même après correction de la vulnérabilité initiale et redéploiement, si ces secrets ne sont pas changés.

6. Synthèse — Les 10 points essentiels appliqués au SGDD

6. Le SGDD est un système distribué : ses cinq services tournent sur des machines/conteneurs distincts, communiquent par messages (HTTP REST et Redis Pub/Sub), et apparaissent à l'utilisateur comme une plateforme documentaire unique.
7. Absence d'horloge globale : les timestamps des logs de chaque service (Auth, Stockage, Recherche...) ne sont pas directement comparables sans synchronisation NTP ; les logs d'audit devront inclure un identifiant de trace (trace ID) pour reconstituer l'ordre causal des événements plutôt que de se fier uniquement aux timestamps.
8. Pannes partielles : la panne du Notification Service ne doit pas empêcher l'upload de documents (traitement asynchrone), alors que la panne d'Auth Service bloque effectivement tout le système — ce qui justifie sa réplication prioritaire.
9. Théorème CAP appliqué : pour la recherche de documents, le SGDD privilégie la disponibilité avec cohérence éventuelle (AP) ; pour l'authentification et les métadonnées de documents, une cohérence plus forte est recherchée (CP).
10. Architecture microservices retenue : flexible et adaptée à un contexte SaaS multi-utilisateurs, au prix d'une complexité opérationnelle (réseau, déploiement, observabilité) plus élevée qu'un simple client-serveur.
11. Idempotence, retry et timeout : l'API Gateway doit configurer des timeouts vers chaque service backend, avec retry et backoff exponentiel pour les erreurs transitoires ; les opérations d'upload devront utiliser une clé d'idempotence pour éviter qu'un retry ne crée un document en double.

12. Surface d'attaque multipliée : sept surfaces d'attaque ont été cartographiées pour seulement cinq services, illustrant la croissance de la surface avec le nombre de composants.
13. Zero Trust : appliqué de façon transversale (AuthN/AuthZ à chaque service, moindre privilège, chiffrement, segmentation, vérification continue), et formalisé via une checklist de readiness.
14. Patterns de résilience utilisés : cache (tokens JWT), file de messages (Stockage → Recherche, Notification), circuit breaker (Gateway → services backend), réplication (bases de données, Auth Service) — chacun avec ses trade-offs assumés.
15. Sécurité by design : le threat modeling STRIDE et la cartographie des surfaces d'attaque ont été réalisés dès la phase de conception de l'architecture, et non en ajout tardif après implémentation.

7. Conclusion

Ce TP nous a permis d'appliquer concrètement les concepts théoriques de la séance (absence d'horloge globale, pannes partielles, théorème CAP, modèles d'architecture, Zero Trust) à la conception d'un cas réel, le Système de Gestion Documentaire Distribué. L'architecture proposée repose sur cinq microservices indépendants derrière un API Gateway, chacun avec sa propre base de données, communiquant majoritairement en synchrone (HTTP REST) mais avec un découplage asynchrone (file de messages) pour les opérations qui ne sont pas sur le chemin critique de l'utilisateur.

L'analyse des défis par composant a montré qu'il n'existe pas de solution universelle : chaque pattern de résilience (cache, réplication, file de messages, circuit breaker) résout un problème tout en introduisant un nouveau compromis, qu'il faut choisir consciemment selon le rôle du composant. La cartographie des surfaces d'attaque et le threat modeling STRIDE ont confirmé que la surface d'attaque croît avec le nombre de services, ce qui justifie une approche Zero Trust systématique plutôt qu'une sécurité concentrée au seul point d'entrée.

Enfin, le scénario de compromission du Stockage Service a illustré l'importance d'un plan de réponse à incident structuré (confinement, éradication, rétablissement, post-mortem) et de la rotation immédiate des secrets après toute compromission suspectée. Ces éléments constituent une base solide pour la séance suivante, consacrée à l'implémentation concrète de la communication entre composants (files de messages, gRPC, pub/sub) sur la base de cette architecture.